

9

杂项讨论

Miscellany

欢迎来到大杂烩的一章。本章只有 3 个条款，但千万别被低微的数字或不迷人的布景愚弄了，它们都很重要！

第一个条款强调不可以轻忽编译器警告信息。至少，如果你希望你的软件有适当行为的话，别太轻忽它们。第二个条款带你综览 C++ 标准程序库，其中覆盖由 TR1 引进的重大新机能。最后一个条款带你综览 Boost，那是我认为最重要的一个 C++ 泛用型网站。如果你尝试写出高效 C++ 软件，却没有参考这些条款所提供的信息，那么充其量也只是一场事倍功半的恶战。

条款 53：不要轻忽编译器的警告

Pay attention to compiler warnings.

许多程序员习惯性地忽略编译器警告。他们认为，毕竟，如果问题很严重，编译器应该给一个错误信息而非警告信息，不是吗？这种想法对其他语言或许相对无害，但在 C++，我敢打赌编译器作者对于将会发生的事情比你有很好的领悟。举个例子，下面是多多少少都会发生在每个人身上的一个错误：

```
class B {
public:
    virtual void f( ) const;
};

class D: public B {
public:
    virtual void f( );
};
```

这里希望以 `D::f` 重新定义 `virtual` 函数 `B::f`, 但其中有个错误: `B` 中的 `f` 是个 `const` 成员函数, 而在 `D` 中它未被声明为 `const`。我手上的一个编译器于是这样说话了:

```
warning: D::f() hides virtual B::f()
```

太多经验不足的程序员对这个信息的反应是: “噢当然, `D::f` 遮掩了 `B::f`, 那正是想象中该有的事!” 错, 这个编译器试图告诉你声明于 `B` 中的 `f` 并未在 `D` 中被重新声明, 而是被整个遮掩了 (条款 33 描述为什么会这样)。如果忽略这个编译器警告, 几乎肯定导致错误的程序行为, 然后是许多调试行为, 只为了找出编译器其实早就侦测出来并告诉你的事情。

一旦从某个特定编译器的警告信息中获得经验, 你将学会了解, 不同的信息意味什么——那往往和它们“看起来”的意义十分不同! 尽管一般认为, 写出一个在最高警告级别下也无任何警告信息的程序最是理想, 然而一旦有了上述的经验和对警告信息的深刻理解, 你倒是可以选择忽略某些警告信息。不管怎样说, 在你打发某个警告信息之前, 请确定你了解它意图说出的精确意义。这很重要。

记住, 警告信息天生和编译器相依, 不同的编译器有不同的警告标准。所以, 草率编程然后倚赖编译器为你指出错误, 并不可取。例如上述发生“函数遮掩”的代码就可能通过另一个编译器, 连半句抱怨和抗议也没有。

请记住

- 严肃对待编译器发出的警告信息。努力在你的编译器的最高 (最严苛) 警告级别下争取“无任何警告”的荣誉。
- 不要过度倚赖编译器的报警能力, 因为不同的编译器对待事情的态度并不相同。一旦移植到另一个编译器上, 你原本倚赖的警告信息有可能消失。

条款 54: 让自己熟悉包括 TR1 在内的标准程序库

Familiarize yourself with the standard library, including TR1.

C++ Standard ——定义 C++ 语言及其标准程序库的规范——早在 1998 年就被标准委员会核准了。标准委员会又于 2003 年发布一个不很重要的“错误修正版”, 并预计于 2008 年左右发布 *C++ Standard 2.0*。日期的不确定性使得人们总是称呼下一版 C++ 为 “C++0x”, 意指 200x 版的 C++。